

Object-Oriented Analysis & Design (OOAD) and UML Cheatsheet

OOAD Core Concepts

Object-Oriented Principles

1. Abstraction

- **Definition:** Simplifying complex reality by modeling classes based on essential properties and behaviors
- **Example:** A `Customer` class abstracts the concept of a customer with essential attributes like name and behaviors like placing orders

2. Encapsulation `np.dot(inputs) ⇒ outputs`

- **Definition:** Hiding internal state and requiring all interaction to occur through an object's methods
- **Benefits:** Reduces complexity, increases reusability, improves maintainability
- **Example:** Making attributes private and providing getters/setters for controlled access

3. Inheritance

- **Definition:** Mechanism where a class inherits properties and methods from a parent class
- **Types:** Single, multiple (in some languages), multilevel, hierarchical
- **Example:** `SavingsAccount` and `CheckingAccount` inherit from `BankAccount`

4. Polymorphism

- **Definition:** Ability of different classes to respond to the same message in different ways

- **Types:**
 - Compile-time (method overloading)
 - Runtime (method overriding)
- **Example:** `Shape.calculateArea()` implemented differently in `Circle` and `Rectangle` classes

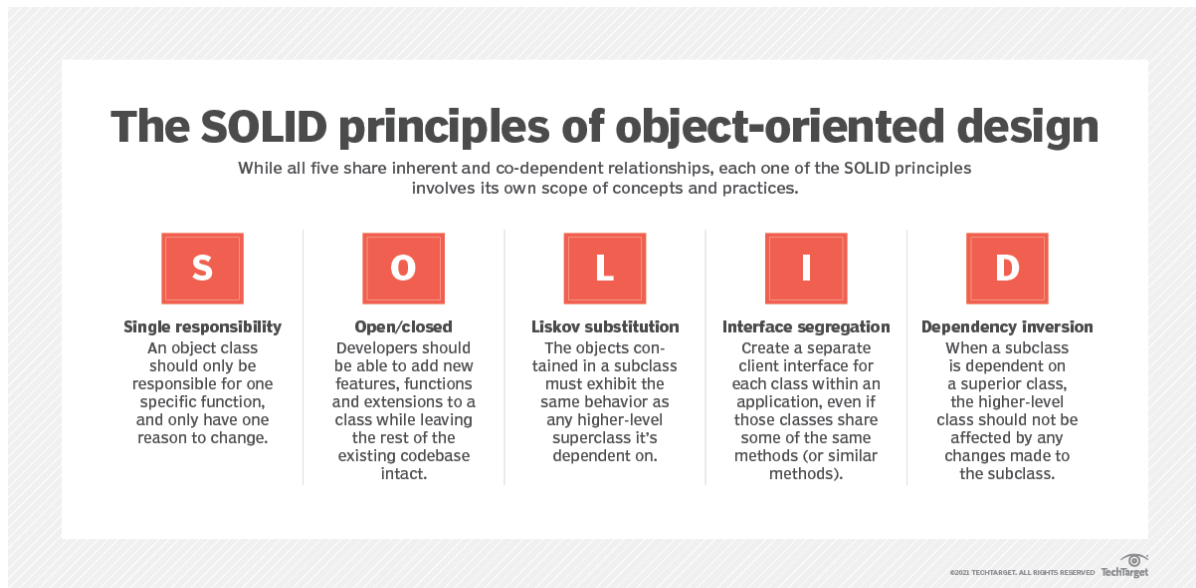
OOAD Process

1. Object-Oriented Analysis (OOA)

- **Purpose:** Identify objects, their attributes, and relationships in the problem domain
- **Deliverables:** Domain model, use cases, initial class diagrams
- **Techniques:**
 - Requirement gathering
 - Use case modeling
 - Domain modeling

2. Object-Oriented Design (OOD)

- **Purpose:** Define software objects and how they collaborate
- **Deliverables:** Detailed class diagrams, sequence diagrams, state diagrams
- **Techniques:**
 - Design patterns
 - SOLID principles
 - CRC cards (Class-Responsibility-Collaboration)



3. Object-Oriented Programming (OOP)

- **Purpose:** Implement the design using an OO programming language
- **Activities:**
 - Creating classes, interfaces
 - Implementing methods
 - Testing objects

SOLID Principles

1. Single Responsibility Principle (SRP)

- A class should have only one reason to change

2. Open-Closed Principle (OCP)

- Software entities should be open for extension but closed for modification

3. Liskov Substitution Principle (LSP)

- Objects of a superclass should be replaceable with objects of a subclass without affecting program correctness

4. Interface Segregation Principle (ISP)

- Many client-specific interfaces are better than one general-purpose interface

5. Dependency Inversion Principle (DIP)

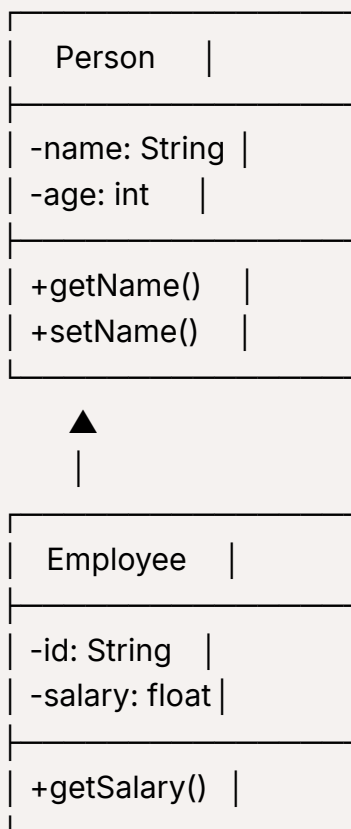
- High-level modules should not depend on low-level modules; both should depend on abstractions

UML Diagrams

Structural Diagrams

1. Class Diagram

- **Purpose:** Shows system's classes, attributes, methods, and relationships
- **Key Elements:**
 - Classes (rectangle with 3 compartments: name, attributes, methods)
 - Relationships (inheritance, association, aggregation, composition)



Relationship Notations:

- **Association:** Basic relationship (—)

- **Inheritance:** "is-a" relationship ($\text{---}\triangleright$)
- **Aggregation:** "has-a" relationship ($\diamond\text{---}$)
- **Composition:** Strong "has-a" relationship ($\blacklozenge\text{---}$)
- **Dependency:** "uses" relationship ($- - \rightarrow$)
- **Realization:** Interface implementation ($- - \triangleright$)

Multiplicity Notations:

- 1: Exactly one
- 0..1: Zero or one
- 0..*: Zero or many
- 1..*: One or many
- n: Exactly n
- n..m: From n to m

2. Object Diagram

- **Purpose:** Shows instances of classes at a specific point in time
- **Key Elements:** Objects (instances) with specific attribute values

3. Component Diagram

- **Purpose:** Shows how components are wired together to form larger components or software systems
- **Key Elements:** Components, interfaces, dependencies

4. Package Diagram

- **Purpose:** Shows how elements are grouped into packages and dependencies between packages
- **Key Elements:** Packages, dependencies

5. Deployment Diagram

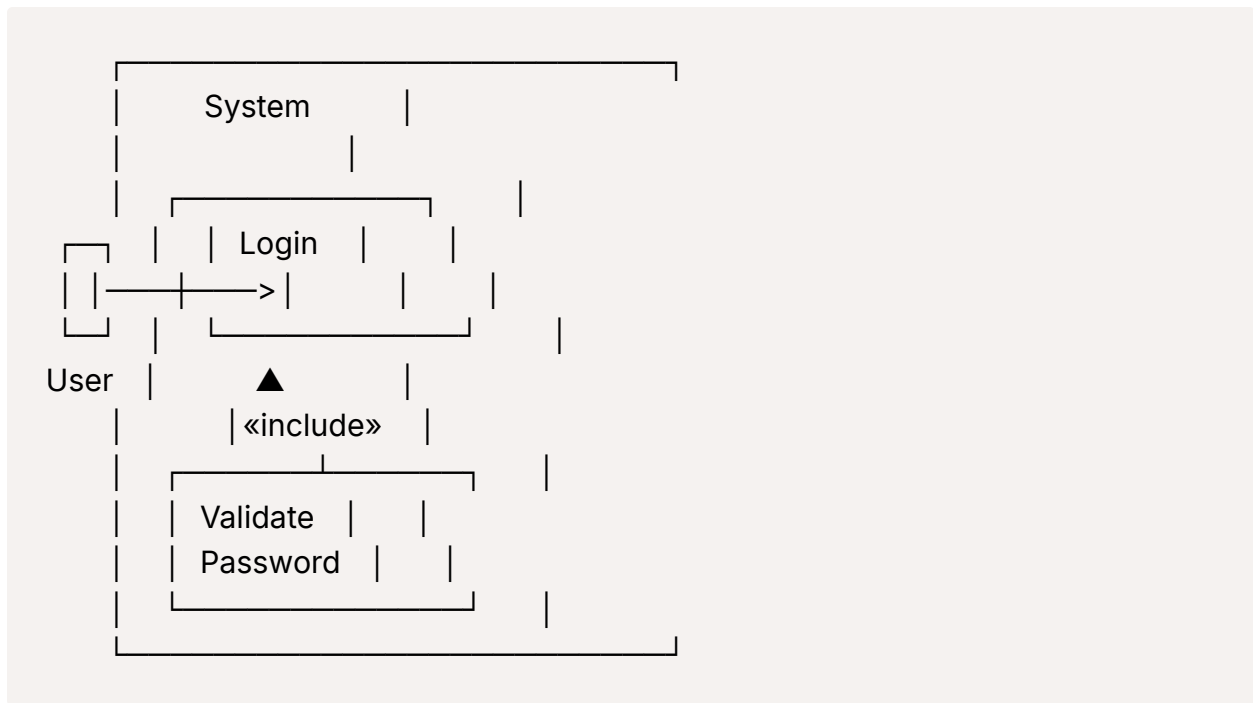
- **Purpose:** Shows hardware configuration and software artifacts deployed on that hardware

- **Key Elements:** Nodes, components, associations

Behavioral Diagrams

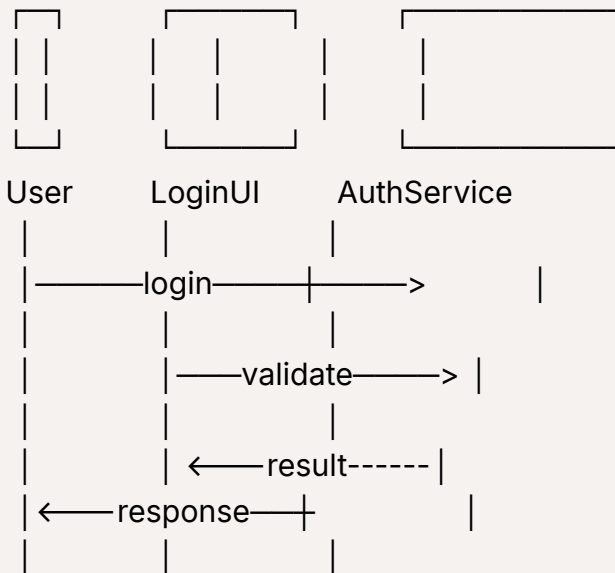
1. Use Case Diagram

- **Purpose:** Shows interactions between users (actors) and the system
- **Key Elements:**
 - Actors (stick figures)
 - Use cases (ovals)
 - Relationships (include, extend, generalization)



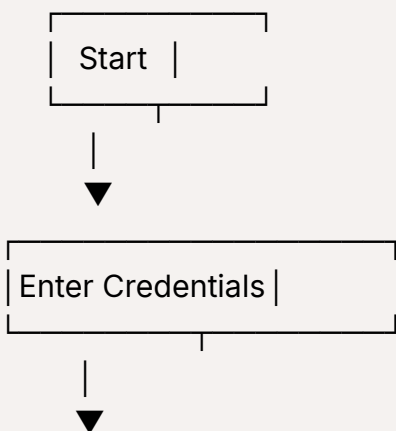
2. Sequence Diagram

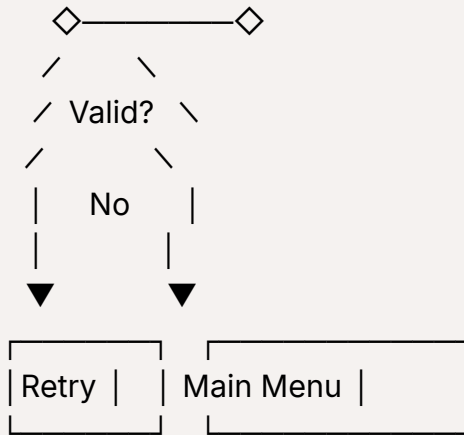
- **Purpose:** Shows object interactions arranged in time sequence
- **Key Elements:**
 - Lifelines (vertical dashed lines)
 - Messages (horizontal arrows)
 - Activation bars (thin rectangles)



3. Activity Diagram

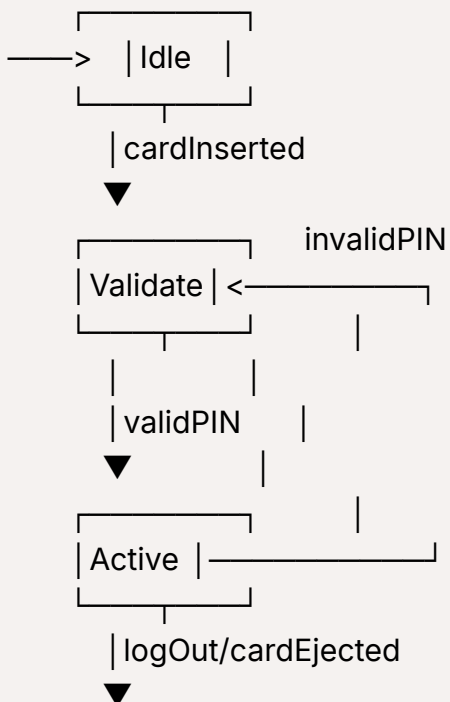
- **Purpose:** Shows flow of control from activity to activity
- **Key Elements:**
 - Activities (rounded rectangles)
 - Transitions (arrows)
 - Decision nodes (diamonds)
 - Fork/join (bars)

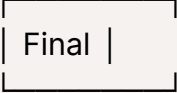




4. State Machine Diagram

- **Purpose:** Shows states of an object and transitions between states
- **Key Elements:**
 - States (rounded rectangles)
 - Transitions (arrows with event labels)
 - Initial/final states (filled circle/circle with dot)





Final |

5. Communication Diagram

- **Purpose:** Shows interactions between objects with numbered sequences
- **Key Elements:** Objects, links, messages with sequence numbers

6. Interaction Overview Diagram

- **Purpose:** Combines activity and sequence diagrams
- **Key Elements:** Activities, interactions

UML Relationships

1. Association

- Represents a relationship between classes
- Indicated with a solid line
- Can be unidirectional or bidirectional
- May include multiplicity and role names

2. Inheritance (Generalization)

- "is-a" relationship
- Child class inherits from parent class
- Represented by a solid line with a hollow arrowhead pointing to the parent

3. Aggregation

- "has-a" relationship (whole-part)
- Weak relationship - parts can exist independently of the whole
- Represented by a solid line with an empty diamond at the whole end

4. Composition

- Stronger form of aggregation

- Parts cannot exist independently of the whole
- Represented by a solid line with a filled diamond at the whole end

5. Dependency

- "uses" relationship
- Represented by a dashed line with an arrow
- Changes in one class may affect the other class

6. Realization

- Class implements an interface
- Represented by a dashed line with a hollow arrowhead

Common OOAD/UML Patterns

Creational Patterns

- **Singleton**: Ensures a class has only one instance
- **Factory**: Creates objects without specifying exact class
- **Builder**: Separates construction from representation
- **Prototype**: Creates objects by cloning existing objects

Structural Patterns

- **Adapter**: Allows incompatible interfaces to work together
- **Composite**: Treats group of objects as a single object
- **Proxy**: Represents another object
- **Decorator**: Adds behavior to objects dynamically

Behavioral Patterns

- **Observer**: Notifies dependents when object changes
- **Strategy**: Defines family of algorithms
- **Command**: Encapsulates request as an object
- **State**: Alters object behavior when state changes

OOAD Process Steps

1. Requirements Analysis

- Gather functional and non-functional requirements
- Create use case diagrams
- Identify key stakeholders and their needs

2. Domain Analysis

- Identify key concepts in the problem domain
- Create a conceptual model
- Develop initial class diagram

3. Interaction Analysis

- Define how objects interact
- Create sequence/collaboration diagrams
- Refine class relationships

4. Detailed Design

- Refine class diagrams with attributes and operations
- Apply design patterns
- Consider non-functional requirements

5. Implementation Planning

- Map design to implementation language
- Consider framework constraints
- Plan testing approach

UML Tools

- **Enterprise Architect** - Comprehensive modeling tool
- **Visual Paradigm** - Full UML support with code generation
- **StarUML** - Open-source UML modeler

- **Lucidchart** - Web-based diagramming tool
- **Draw.io** - Free online diagram software
- **PlantUML** - Text-based UML diagram creation

Best Practices

Analysis Phase

- Focus on what, not how
- Understand the domain thoroughly
- Get stakeholder involvement early
- Keep models simple initially

Design Phase

- Apply SOLID principles
- Consider design patterns
- Design for change
- Balance flexibility with complexity

UML Modeling

- Use consistent notation
- Keep diagrams focused on specific aspects
- Don't overmodel - use diagrams that add value
- Update models as design evolves

Flask Cheatsheet

Here's a comprehensive Flask cheatsheet to help you develop web applications with this Python microframework:

Basic Flask Application

```
from flask import Flask, render_template, request, redirect, url_for, session, flash

# Initialize the app
app = Flask(__name__)
# Basic route
@app.route('/hello')
def index():
    return 'Hello, World!'

# Run the app
if __name__ == '__main__':
    app.run(debug=True)
```

Routing

```
# Basic route
@app.route('/path')
def path_handler():
    return 'This is a path'

# Route with variables
@app.route('/user/<username>')
def show_user(username):
    return f'User: {username}'

# Route with type constraints
@app.route('/post/<int:post_id>')
def show_post(post_id):
    return f'Post ID: {post_id}'
```

```
# Routes with HTTP methods
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        # Process form data
        return 'Form submitted'
    else:
        # Show form
        return 'Please log in'
```

Templates

```
@app.route('/template')
def template_example():
    return render_template('example.html', title='Flask', message='Hello!')
```

```
<!-- templates/example.html →
<!DOCTYPE html>
<html>
<head>
  <title>{{ title }}</title>
</head>
<body>
  <h1>{{ message }}</h1>
  {% if user %}
    <p>Welcome, {{ user.name }}!</p>
  {% else %}
    <p>Please log in.</p>
  {% endif %}

  {% for item in items %}
    <p>{{ item }}</p>
  {% endfor %}
```

```
</body>
</html>
```

Request Handling

```
@app.route('/form', methods=['POST'])
def handle_form():
    username = request.form.get('username')
    password = request.form.get('password')

    # Get query parameters
    page = request.args.get('page', 1, type=int)

    # Get JSON data
    data = request.json

    # Get files
    if 'file' in request.files:
        file = request.files['file']
        file.save('/path/to/save/' + file.filename)
```

Redirects and Errors

```
app.route('/redirect')
def redirect_example():
    return redirect(url_for('index'))

@app.route('/error')
def error_example():
    return render_template('error.html'), 404# Set status code

# Custom error handler
@app.errorhandler(404)
```

```
def page_not_found(e):  
    return render_template('404.html'), 404
```

Sessions and Cookies

```
@app.route('/session')  
def session_example():  
    session['username'] = 'user123'# Set session variable  
    user = session.get('username')# Get session variable  
    session.pop('username', None)# Delete session variable  
  
    # Set cookies  
    resp = make_response(render_template('index.html'))  
    resp.set_cookie('user_id', '12345')  
  
    # Get cookies  
    user_id = request.cookies.get('user_id')  
  
    return resp
```

Flask Blueprints

```
# In blueprints/admin.py  
from flask import Blueprint, render_template  
  
admin = Blueprint('admin', __name__, url_prefix='/admin')  
  
@admin.route('/')  
def admin_index():  
    return render_template('admin/index.html')  
  
# In main app.py
```



```
from blueprints.admin import admin
app.register_blueprint(admin)
```

Database with Flask-SQLAlchemy

```
from flask_sqlalchemy import SQLAlchemy

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database.db'
db = SQLAlchemy(app)

# Define a model
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

    def __repr__(self):
        return f'<User {self.username}>'

# Create the database and tables
with app.app_context():
    db.create_all()

# CRUD operations
@app.route('/create_user')
def create_user():
    new_user = User(username='john', email='john@example.com')
    db.session.add(new_user)
    db.session.commit()
    return 'User created!'

@app.route('/get_users')
def get_users():
    users = User.query.all()
    user = User.query.filter_by(username='john').first()
```

```

    return str(users)

@app.route('/update_user/<int:id>')
def update_user(id):
    user = User.query.get(id)
    user.username = 'updated_name'
    db.session.commit()
    return 'User updated!'

@app.route('/delete_user/<int:id>')
def delete_user(id):
    user = User.query.get(id)
    db.session.delete(user)
    db.session.commit()
    return 'User deleted!'

```

Authentication with Flask-Login

```

from flask_login import LoginManager, UserMixin, login_user, logout_user, login_re
quired, current_user

login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True)
    password = db.Column(db.String(120))

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':

```

```

        user = User.query.filter_by(username=request.form['username']).first()
        if user and user.password == request.form['password']:# Use proper password hashing!
            login_user(user)
            return redirect(url_for('protected'))
        return render_template('login.html')

@app.route('/logout')
def logout():
    logout_user()
    return redirect(url_for('index'))

@app.route('/protected')
@login_required
def protected():
    return f'Hello, {current_user.username}!'

```

Deployment Configuration

```

# Development
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)

# Production with Gunicorn# Run: gunicorn -w 4 -b 0.0.0.0:8000 app:app# Environment variables
import
app.config['DATABASE_URL'] = os.environ.get('DATABASE_URL', 'sqlite:///default.db')

```

Flask Extensions

```

from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Email

class LoginForm(FlaskForm):

```

```

email = StringField('Email', validators=[DataRequired(), Email()])
password = PasswordField('Password', validators=[DataRequired()])
submit = SubmitField('Log In')

# Flask-Migrate for database migrations
from flask_migrate import Migrate
migrate = Migrate(app, db)
# Run: flask db init, flask db migrate, flask db upgrade# Flask-RESTful for APIs
from flask_restful import Api, Resource
api = Api(app)

class UserResource(Resource):
    def get(self, user_id):
        user = User.query.get(user_id)
        return {'username': user.username}

api.add_resource(UserResource, '/api/user/<int:user_id>')

```